

比较，需要更高的性能、更少的接口和更高的交换速率。

网络主干部分称为核心层，核心层的主要目的在于通过高速转发通信，提供优化、可靠的骨干传输结构，因此，核心层交换机应拥有更高的可靠性、性能和吞吐量。核心层为网络提供了骨干组件或高速交换组件，在纯粹的分层设计中，核心层只完成数据交换的特殊任务。用户需要根据网络需求的地理距离、信息流量和数据负载的轻重来选择核心层技术，常用的技术包括 ATM、100Base-FX 和千兆以太网等。在主干网中，考虑到高可用性的需求，通常会使用双星（树）结构，即采用两台同样的交换机，与汇聚层交换机分别连接，并使用链路聚合技术实现双机互连。

4.4.3 网络实施

网络实施是在网络设计的基础上进行设备的购买、安装、调试和系统切换工作。网络实施主要包括以下步骤：

（1）工程实施计划。在网络设备安装前，需要编制工程实施计划，列出需实施的项目、费用和负责人等，以便控制投资，按进度要求完成实施任务。工程实施计划必须包括在网络实施阶段的设备验收、人员培训、系统测试和网络运行维护等具体事务的处理，必须控制和处理所有可预知的事件，并调动有关人员的积极性。

（2）网络设备到货验收。系统中要用到的网络设备到货后，在安装调试前，必须先进行严格的功能和性能测试，以保证购买的产品能很好地满足用户需要。在到货验收的过程中，要做好记录，包括对规格、数量和质量进行核实，以及检查合格证、出厂证、供应商保证书和各种证明文件是否齐全。在必要时利用测试工具进行评估和测试，评估设备能否满足网络建设的需求。如果发现短缺或破损，要求设备提供商补发或免费更换。

（3）设备安装。网络系统的安装和调试需要由专门的技术人员负责。安装项目一般分为综合布线系统、机房工程、网络设备、服务器、系统软件和应用软件等几个部分，不同的部分应分别由专门的工程师进行安装和调试。在这些安装项目中，尤其要注意综合布线系统的质量，因为综合布线一般会涉及隐蔽工程，一旦覆盖后发生故障，查找错误源和恢复故障的代价比较高。

（4）系统测试。系统安装完毕，就要进行系统测试。系统测试是保证网络安全可靠运行的基础。网络测试包括网络设备测试、网络系统测试和网络应用测试三个层次。网络设备测试主要是针对交换机、路由器、防火墙和线缆等传输介质和设备的测试；网络系统测试主要是针对系统的连通性、链路传输率、吞吐率、传输时延和丢包率、链路利用率、错误率、广播帧及组播帧及冲突率等方面的测试；网络应用测试主要是针对 DHCP、DNS、Web、E-mail 和 FTP 等服务性能进行测试。

（5）系统试运行。系统调试完毕后，进入试运行阶段。这一阶段是验证系统在功能和性能上是否达到预期目标的重要阶段，也是对系统进行不断调整，直至达到用户要求的重要时刻。

（6）用户培训。一个规模庞大、结构复杂的网络系统往往需要网络管理员来维护，并协调网络资源的使用。对有关人员的培训是网络建设的重要一环，也是保证系统正常运行的重要因素之一。

(7) 系统转换。经过一段时间的试运行，系统达到稳定、可靠的水平，就可以进行系统转换工作。系统转换可以采用三种方法：直接转换、并行转换和分段转换。这三种方法的可靠性和成本各不相同，应视具体情况而定。

4.5 分布式系统

分布式系统是其组件分布在联网的计算机上，组件之间通过消息传递进行通信和动作协调的系统。

分布式系统的关键特征是组件的并发性（进程间并发）、缺乏全局时钟和组件故障的独立性等。构建和使用分布式系统的主要动力来源于对共享资源的期望。这里的“资源”是抽象的概念，但它能很好地描述在互联计算机间共享事物的范围。如这里的“资源”可以是硬件（磁盘、打印机等），也可以是软件（如文件、数据库、应用服务等）。

构建分布式系统的挑战是处理其组件的异构性（Heterogeneity）、开放性（Openness）（允许增加或替换组件）、安全性（Security）、可伸缩性（Scalability）（用户的负载或数量增加时，系统能正常运行的能力）、故障处理（Failure handling）、并发性（Concurrency）、透明性（Transparency）和服务质量（Quality of Service, QoS）。

1) 异构性

互联网使得用户能在大量异构计算机和网络上访问服务和运行应用程序。

虽然互联网由多种不同种类的网络组成，但所有连接到互联网的设备都使用相同的网络协议来通信，所以这些不同网络的区别被屏蔽了。

整型等数据类型在不同种类的硬件上可以有不同的表示方法。如数据存储方法有大端存储和小端存储。一般情况下，高级编程语言会屏蔽这部分差异。

虽然计算机的操作系统需要包含互联网协议的实现，但不为这些协议提供相同的接口实现。如 Linux 中消息交换的调用与 Windows 中的调用是不一样的（面向接口编程）。

不同的编程语言使用不同的方式表示字符和数据结构（如数组和对象）。如果想让这些语言编写的程序进行通信，必须使用公共的标准。如远程过程通信协议 gRPC 等。

2) 开放性

计算机系统的开放性是决定系统能否以不同方式被扩展和重新实现的特征。分布式系统的开放性主要取决于新的资源共享服务能被增加和供多种客户程序使用的程度。

除非软件开发能获得系统组件的关键软件接口的规约和文档，否则无法实现开放性。解决方法是发布关键接口。这个过程类似于接口的标准化，但它经常避开官方的标准化过程（官方的标准化过程通常烦琐且进度缓慢）。

然而，发布接口仅是分布式系统增加和扩展服务的起点。设计者所面临的挑战是解决由不同人构造的、由许多组件组成的分布式系统的复杂性。

互联网协议的设计者引入了一系列称为“征求意见稿”（Request For Comments, RFC）的文档。该文档包含规约和讨论。

按上述方式支持资源共享的系统称为开放分布式系统。之所以具备开放性，主要是强调它们是可扩展的。可以通过在网络中增加计算机实现在硬件层次上的扩展，通过引入新的服务、重新实现旧的服务实现在软件层次上的扩展，最终使得应用能共享资源。开放分布式系统有以下特点：

(1) 发布系统的关键接口。

(2) 基于一致的通信机制和发布接口访问共享资源。

(3) 可以基于异构硬件和软件构造系统。注意，如果期望系统正常工作，相关的测试是必要的。

3) 安全性

分布式系统中维护和使用的众多信息资源对用户具有很高的内在价值，因此这些信息资源的安全相当重要。信息资源的安全性包括三个部分：机密性（防止泄露未经授权的个人数据）、完整性（防止被改变或破坏）、可用性（防止对访问资源的干扰，如 DDoS 攻击）。

4) 可伸缩性

分布式系统可以在不同的规模（从小型企业内部网络到互联网）下有效且高效运转。如果资源数量和用户数量激增，系统仍能保持其有效性，那么这个系统称为可伸缩的。

理想状态下，系统规模增加时系统和应用应不需要随之改变，但这一点很难达到。规模问题是分布式系统开发中面临的主要问题。

5) 故障处理

计算机系统有时会出现故障。当硬件或软件发生故障时，程序可能会产生不正确的结果，或者在它们完成应进行的计算之前停止了。分布式系统的故障是部分的，也就是说，有些组件出了故障，而有些组件运行正常。因此故障的处理相当困难。常见的处理故障的技术有：

(1) 检测故障。有些故障可以被检测到，如校验和可用于检测消息或文件中出错的数据。

(2) 掩盖故障。有些被检测到的故障能被隐藏起来或降低它的严重程度。如消息在不能到达时进行重传；将文件数据写入两个磁盘，如果一个磁盘坏掉，另一个磁盘的数据仍是正确的。

(3) 容错。互联网上的大多数服务确实可能发生故障，试图检测并隐藏所有故障是不现实的。服务的客户能被设计成容错的，这通常也涉及客户的错误容忍度。如 Web 服务器不可用时，Web 浏览器不会让用户一直等待，而是告知用户问题原因。

(4) 故障恢复。恢复涉及软件的设计，以便在服务器崩溃后，永久数据的状态能被恢复或回滚。如数据库领域，通过实现事务保证单机场景下的故障恢复。而分布式事务则保证多机场景下的事务。

6) 并发性

在分布式系统中，服务和应用均提供可被客户共享的资源。因此，可能有几个客户同时试图访问一个共享资源的情况。管理共享资源的进程可以一次接收一个客户请求，但这种方法限制了吞吐量。因此，服务和应用通常允许并发地处理多个客户请求。如为避免进程间的竞争，通常可使用分布式锁保证资源服务的顺序性。为避免一个进程中多个线程间的竞争，可使用不同粒度的锁（读锁、写锁、公平锁、非公平锁）来保证数据的一致性。

7) 透明性

透明性被定义为对用户和开发人员屏蔽分布式系统的组件分离性，使系统被认为是一个整体，而不是独立组件的集合。透明性的含义对系统软件的设计有重大影响。

ANSA 参考手册和国际标准化组织的开放分布式处理的参考模型（RM-ODP）识别出 8 种透明性。下面将解释最初的 ANSA 定义，并使用移动透明性（使用更加广泛）替换迁移透明性。

（1）访问透明性（Access Transparency）：用相同的操作访问本地资源和远程资源，对不同数据表示形式及资源访问方式进行隐藏。

（2）位置透明性（Location Transparency）：为资源分配逻辑名，用户不需要知道资源的物理位置或网络位置就能访问它们。

（3）并发透明性（Concurrency Transparency）：几个进程能并发地使用共享资源进行操作且互不干扰，即对共享资源的并发访问不会破坏资源的一致状态。

（4）复制透明性（Replication Transparency）：使用资源的多个实例提升可靠性和性能，而用户和开发人员无须知道副本的相关信息。

（5）故障透明性（Failure Transparency）：屏蔽错误，无论是硬件故障还是软件故障，用户不会注意到资源的故障以及系统对于该故障的恢复过程。

（6）移动透明性（Mobility Transparency）：资源和客户能够在系统内移动而不会影响用户或程序的操作。

（7）性能透明性（Performance Transparency）：当负载变化时，系统能被重新配置以适应性能。

（8）伸缩透明性（Scaling Transparency）：系统和应用能够进行扩展而不改变系统结构或应用算法。

有时，也将访问透明性和位置透明性统称为网络透明性。

8) 服务质量

一旦给用户提供他们要求的服务的功能，如在一个分布式系统中的文件服务，人们就能继续探究所提供的服务质量。系统的主要非功能特性，即影响客户和用户体验的服务质量是可靠性、安全性和性能。

可靠性和安全性问题在设计大多数计算机系统时是关键的。服务质量的性能方面源于及时性和计算吞吐量，但它已被重定义成满足及时性保证的能力。

典型的分布式系统类型有分布式文件系统、分布式数据库、分布式缓存系统、分布式协调服务系统、分布式计算框架等。

1. 分布式文件系统

分布式存储最早是由谷歌提出的概念，其目的是通过廉价的服务器为大规模、高并发场景下的网页访问场景提供服务。分布式存储扩展能力强，容量随节点数线性上涨，可达到 EB（亿字节）级别，但网络时延会成为分布式存储对外提供服务时的瓶颈。

目前主流的分布式文件系统（Distributed File System）有 GFS（Google File System）、

HDFS、TFS（Taobao File System）、Ceph、Lustre、MogileFS、MooseFS、FastDFS、GridFS 等。

(1) GFS 是 Google 公司为了满足本公司需求而开发的基于 Linux 的专有分布式文件系统，它运行于廉价的普通硬件上，并提供容错功能。它可以给大量的用户提供总体性能较高的服务。尽管 Google 公布了该系统的一些技术细节，但 Google 并没有将该系统的软件部分作为开源软件发布。

(2) Hadoop 分布式文件系统（HDFS）是一个具备高度容错性的系统，适合部署在廉价的机器上，系统使用多台计算机存储文件，并且提供统一的访问接口，像是访问一个普通文件系统一样。HDFS 采用主从（Master/Slave）结构模型，一个 HDFS 集群是由一个 NameNode 和若干个 DataNode 组成的。其中 NameNode 作为主服务器，管理文件系统的命名空间和客户端对文件的访问操作，集群中的 DataNode 管理存储的数据。

(3) TFS 是一个高可扩展、高可用、高性能、面向互联网服务的分布式文件系统，主要针对海量的非结构化数据，它构筑在普通的 Linux 机器集群上，可为外部提供高可靠和高并发的存储访问。TFS 为淘宝提供海量小文件存储，通常文件大小不超过 1MB，满足了淘宝对小文件存储的需求，被广泛地应用在淘宝各项应用中。它采用了 HA 架构和平滑扩容，保证了整个文件系统的可用性和扩展性。同时，扁平化的数据组织结构可将文件名映射到文件的物理地址，简化了文件的访问流程，在一定程度上为 TFS 提供了良好的读写性能。

(4) MooseFS 是一款相对小众的分布式文件系统，不需要修改上层应用接口即可直接使用。MooseFS 由管理服务器（Master Server）、数据服务器（Chunk Server）、元数据备份服务器（Metalogger Server）和访问 MooseFS 中文件的客户机（Client Computers）4 部分组成。

管理服务器负责各个数据服务器的管理、文件读写调度、文件空间回收及恢复，可多节点拷贝。

数据服务器负责连接管理服务器，听从管理服务器调度，提供存储空间，并为客户提供数据传输。

元数据备份服务器可以是任何数量的服务器，负责备份管理服务器的变化日志文件，文件类型为 changelog_ml.*.mfs，以便于在管理服务器宕机时接替其进行工作。

访问 MooseFS 中文件的客户机是任意数量的机器，使用 mfsmount 过程与管理服务器进行通信（接收和修改文件元数据）和块服务器（实际的文件数据交换）。

mfsmount 是基于 FUSE 机制（文件系统用户空间）的，所以 MooseFS 对所有采用 FUSE 实现机制的操作系统（Linux、FreeBSD、MacOS X 等）都是有效的。

元数据存储和管理服务器的内存中，同时还保存至磁盘（定期更新二进制文件，并且立即更新增量日志）。作为日志的主二进制文件也将被同步到元数据日志服务器（metalloggers）（如果存在的话）。

2. 分布式缓存系统

分布式缓存系统是指缓存数据分布在不同的服务器上，通过缓存技术实现数据的快速访问和高并发的请求处理。

分布式缓存的典型应用是缓存经过复杂计算得到的数据，或者缓存系统中频繁访问的热点数据，以减轻数据库压力。

常见的分布式缓存系统有 Memcache、Redis、Tair 等。

1) Memcache

不同分布式缓存的实现原理通常有比较大的差异，Memcache 通过在内存里维护一个统一且巨大的哈希（Hash）表，来提高读取速度，它能够用来存储各种格式的数据，包括图像、视频、文件以及数据库检索的结果等。

Memcache 使用物理内存作为缓存区，使用键值（key/value）对的方式来存储数据，可独立运行在服务器上。每个进程最大 2GB，如果想缓存更多的数据，可以开辟更多的 Memcache 进程（不同端口）或者使用分布式 Memcache 进行缓存，将数据缓存到不同的物理机或者虚拟机上。

Memcache 采用 LRU 缓存淘汰策略。在 Memcache 内存存储数据项时，可以指定其缓存的失效时间，默认为永久。当 Memcache 服务器用完分配的内存时，失效的数据被首先替换，然后是最近未使用的数据。在 LRU 中，Memcache 使用了一种 Lazy Expiration 策略，它不会监控存入的键/值（key/value）对是否过期，而是在获取 key 值时查看记录的时间戳，检查 key/value 对空间是否过期，以减轻服务器的负载。

Memcache 不支持持久化，重启服务会导致数据丢失。

Memcache 服务器之间彼此不通信，其分布式能力是依赖客户端来实现的。具体来说，就是在客户端实现一种算法，根据 key 计算出数据应该向哪个服务器节点读/写。选取集群节点的常见算法如下：

（1）哈希取余算法。通过公式 $\text{Hash}(\text{key}) \% N$ 计算出数据映射到的节点。

（2）一致性哈希算法。将所有的存储节点排列在首尾相接的 Hash 环上，每个 key 在计算出 Hash 后按照顺时针方向找到邻接的存储节点存放。当有节点加入或退出时，仅影响该节点在 Hash 环上顺时针方向相邻的后续节点。

（3）虚拟 Hash 槽算法。使用散列性能良好的哈希函数把所有数据映射到一个固定范围的整数集合中，整数定义为槽（slot），这个范围一般远远大于节点数。槽是集群内数据管理和迁移的基本单位。采用大范围槽的主要目的是方便数据拆分和集群扩展。每个节点会负责一定数量的槽。

2) Redis

Redis 是一个开源的高性能的键值对存储系统。Redis 支持多种数据结构，包括字符串、哈希、列表、集合和有序集合等。

Redis 的分布式缓存技术使用了一种称为“一致性哈希”的算法。该算法将缓存分散在多个节点上，每个节点存储部分缓存数据。当需要读取或写入数据时，客户端会将数据的键值通过哈希函数计算出一个哈希值，然后根据这个哈希值选择一个节点进行读写操作。

Redis 的特点是快速、可扩展、高效和可靠，可以处理高并发的读写请求。

Redis 的优点包括高性能读写，多数据类型支持，数据持久化，高可用架构，支持自定义虚拟内存，支持分布式分片集群，单线程读写性能极高。

Redis 的缺点是多线程读写较 Memcache 慢。